Министерство образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Динамические списки»

Выполнили:

студенты группы 25ВВВ3

Шибанов Ф.Д.

Флеонов М.В.

Пенза 2026

Цель работы: Изучить динамические списки как способ хранения данных, объём которых заранее неизвестен, освоить операции создания элемента, добавления, просмотра, поиска и удаления элементов односвязного списка и на его основе реализовать приоритетную очередь, очередь и стек.

Описание алгоритмов

Односвязный список.

Динамический список представляет собой последовательность элементов, память под которые выделяется во время работы программы по мере необходимости. Каждый элемент описывается структурой из информационного поля и указателя на следующий элемент; у последнего элемента этот указатель равен NULL. Обращение к списку выполняется через указатель на первый элемент (голову), а для быстрого добавления в конец дополнительно хранится указатель на последний элемент. Прямого доступа к элементу по номеру нет: чтобы добраться до нужного элемента, список просматривается последовательно от головы. За гибкость распределения памяти, таким образом, приходится платить эффективностью доступа.

Создание элемента.

Функция создания элемента выделяет функцией malloc память под структуру узла; при неудаче выводится сообщение и программа завершается. Затем с клавиатуры вводятся данные, которые копируются в информационное поле, указателю на следующий элемент присваивается NULL, и функция возвращает указатель на созданный узел. Если запись не произведена, возвращается NULL. В отличие от кода методических указаний, в этом случае выделенная память освобождается, что исключает её утечку, а приведение типа записано в виде (struct node*), поскольку программа компилируется компилятором языка C.

Добавление элемента в конец списка.

Если список пуст, новый элемент становится одновременно головой и последним элементом. Иначе поле next последнего элемента связывается с новым узлом, после чего указатель на последний элемент переносится на него. Благодаря хранению этого указателя добавление выполняется за постоянное время без просмотра списка.

Просмотр списка.

Вспомогательный указатель устанавливается на голову и перемещается по полям next до значения NULL; на каждом шаге выводится информационное поле текущего элемента. Время просмотра пропорционально длине списка.

Поиск и удаление элемента.

Поиск по содержимому выполняет последовательный просмотр списка со сравнением строк функцией strcmp и возвращает указатель на найденный элемент либо NULL. При удалении дополнительно запоминается указатель на предыдущий элемент: удаляемый узел исключается из цепочки переназначением поля next предыдущего элемента, а при удалении первого элемента переносится голова списка. Память исключённого узла освобождается функцией free.

Ход работы

Задание 1

Задание: реализовать приоритетную очередь путём добавления элемента в список в соответствии с приоритетом объекта (объект с большим приоритетом становится перед объектом с меньшим приоритетом).

В структуру узла добавлено целочисленное поле приоритета, и функция создания элемента дополнительно запрашивает его значение. Функция добавления помещает элемент не в конец, а в позицию, соответствующую приоритету, поэтому список всё время остаётся упорядоченным и элемент с наивысшим приоритетом находится в его голове. Рассматриваются три случая. Если список пуст, элемент становится головой и последним элементом. Если приоритет нового элемента выше приоритета головы, элемент вставляется перед ней и становится новой головой. В остальных случаях список просматривается двумя указателями, на текущий и на предыдущий элементы, до первого элемента с более низким приоритетом, и новый узел вставляется между ними. Если вставка пришлась на конец списка, обновляется указатель на последний элемент.

В реализации принято, что меньшее число означает более высокий приоритет (приоритет 1 — наивысший), как при нумерации очерёдности, поэтому элементы в списке располагаются по возрастанию числового значения приоритета. При равенстве приоритетов новый элемент помещается после уже имеющихся, то есть элементы одного приоритета обслуживаются в порядке поступления. Извлечение элемента с наивысшим приоритетом сводится к взятию головы списка и выполняется за постоянное время, тогда как добавление требует просмотра списка и в худшем случае занимает время, пропорциональное его длине.

Работа программы показана на рисунке 1: элементы введены в порядке oleg (3), maxim (2), fedor (1), а при просмотре выведены в порядке убывания приоритета.

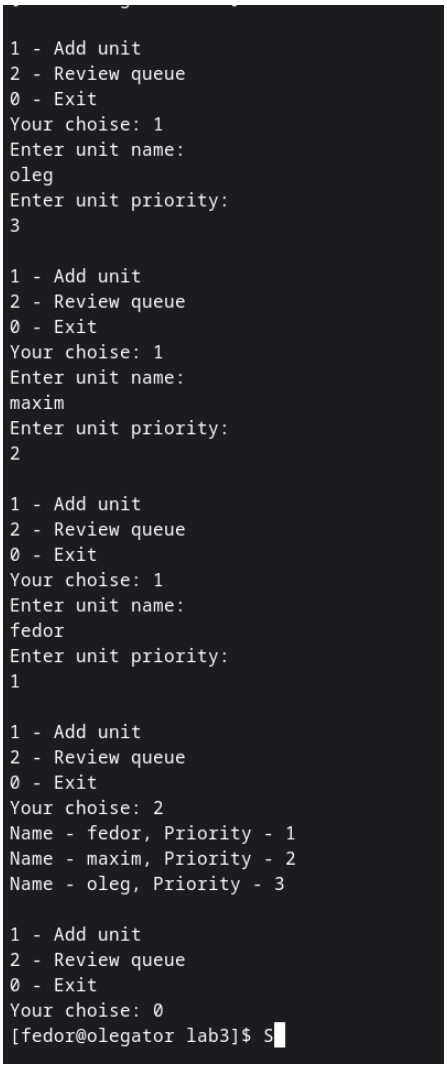


Рисунок 1 — Работа программы «Приоритетная очередь»

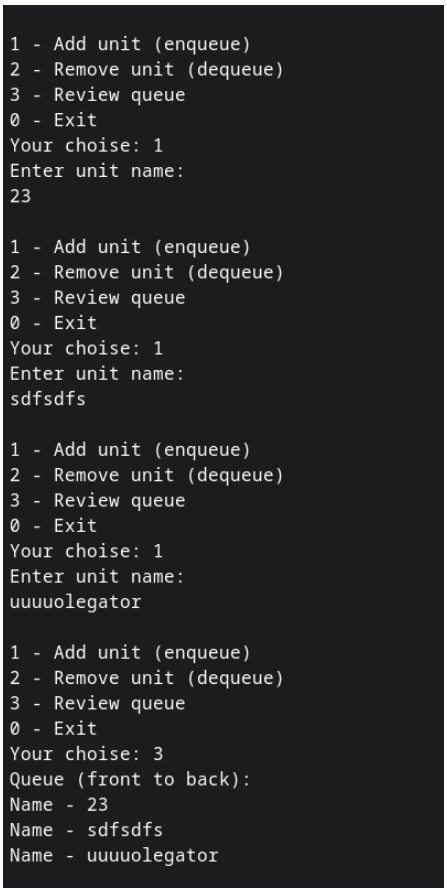
Задание 2

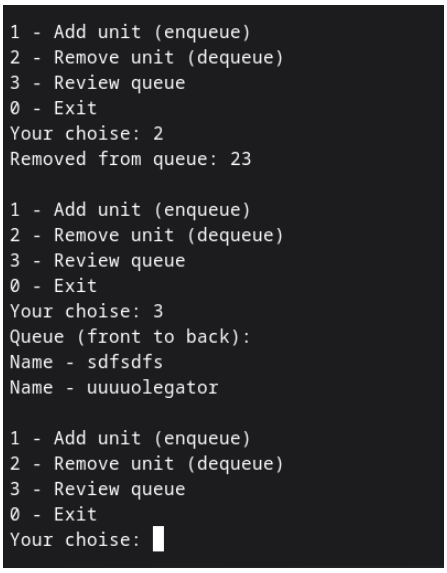
Задание: на основе приведённого кода реализовать структуру данных «Очередь».

Очередь работает по принципу FIFO («первым пришёл — первым ушёл»): элементы добавляются в конец, а извлекаются из начала. Функция добавления (enqueue) повторяет исходную функцию добавления в конец списка с использованием указателя на последний элемент. Функция извлечения (dequeue) проверяет, не пуста ли очередь, выводит информационное поле головы, переносит голову на следующий элемент и освобождает память извлечённого узла. Если после извлечения очередь становится пустой, указатель на последний элемент также обнуляется: в противном случае он указывал бы на освобождённую память, и следующее добавление привело бы к ошибке. Обе

операции выполняются за постоянное время. В меню программы добавлен пункт извлечения элемента.

Работа программы показана на рисунке 2: после добавления трёх элементов извлекается первый введённый — 23, остальные сохраняют порядок поступления.





а) б)

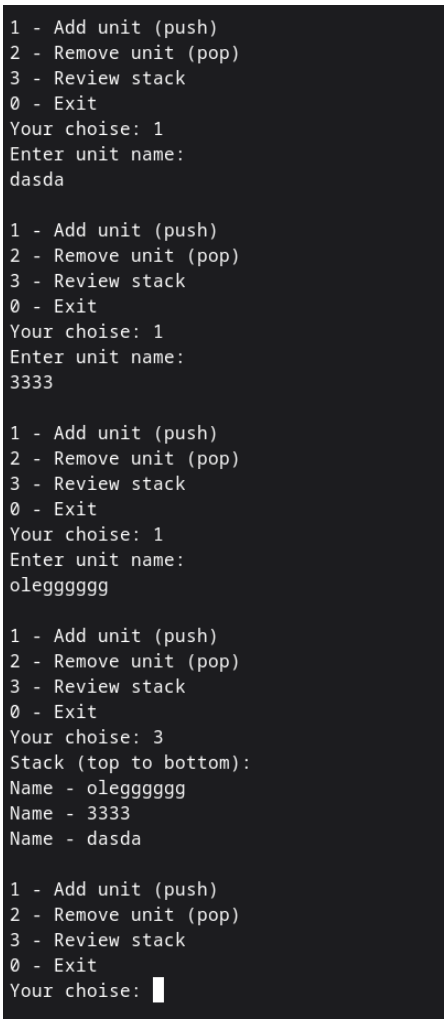
Рисунок 2 — Работа программы «Очередь»: а) добавление и просмотр элементов; б) извлечение элемента

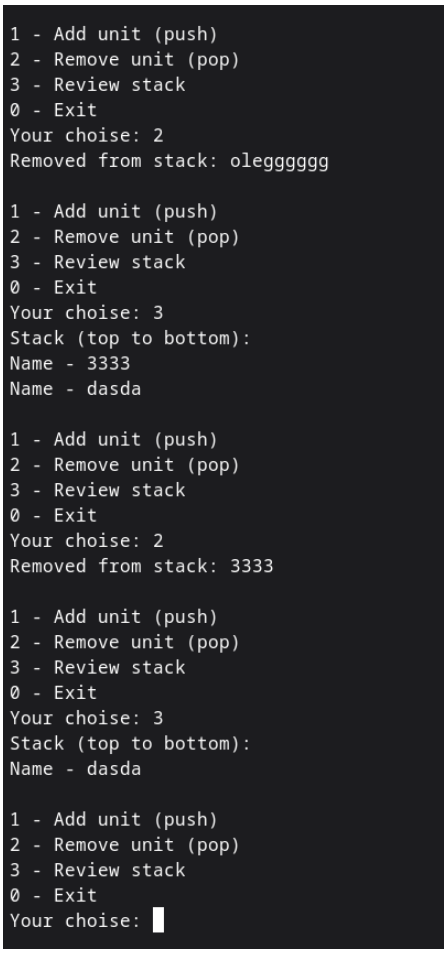
Задание 3

Задание: на основе приведённого кода реализовать структуру данных «Стек».

Стек работает по принципу LIFO («последним пришёл — первым ушёл»): все операции выполняются с одним концом списка, вершиной, которой служит голова. При добавлении (push) поле next нового элемента связывается с текущей головой, после чего голова переносится на новый элемент. При извлечении (pop) выводится и удаляется головной элемент, а голова переносится на следующий. Указатель на последний элемент стеку не нужен и из программы исключён. Обе операции выполняются за постоянное время и не зависят от числа элементов.

Работа программы показана на рисунке 3: элементы извлекаются в порядке, обратном порядку добавления: сначала удалён последний введённый элемент, затем предпоследний.





а) б)

Рисунок 3 — Работа программы «Стек»: а) добавление и просмотр элементов; б) извлечение элементов

Характеристики реализованных структур сведены в таблицу 1.

Таблица 1 — Структуры данных на основе односвязного списка

Структура	**Добавление**	**Извлечение**	**Порядок обслуживания**
Приоритетная очередь	в позицию по приоритету, O(n)	из начала, O(1)	по приоритету, при равенстве — по поступлению
Очередь	в конец, O(1)	из начала, O(1)	

Стек	в начало, O(1)	из начала, O(1)	

Из таблицы видно, что очередь и стек отличаются лишь тем, с какого конца списка выполняется добавление, а приоритетная очередь переносит затраты на момент вставки, чтобы извлечение оставалось быстрым.

После выполнения заданий были скомпилированы исполняемые файлы. Изменения закоммичены и загружены на GitHub.

Ссылка на репозиторий

Вывод: В ходе лабораторной работы изучены динамические односвязные списки и операции над ними: создание элемента с выделением памяти, добавление, просмотр, поиск и удаление. На основе списка реализованы три структуры данных. Приоритетная очередь получена изменением функции добавления: элемент вставляется в позицию согласно приоритету, благодаря нему список остаётся упорядоченным, а элемент с наивысшим приоритетом всегда находится в голове. Очередь реализована добавлением в конец и извлечением из начала списка, стек — добавлением и извлечением в начале списка. Установлено, что для очереди и стека обе операции выполняются за постоянное время, а вставка в приоритетную очередь требует просмотра списка и имеет сложность O(n), что компенсируется мгновенным извлечением элемента с наивысшим приоритетом.